1. ISAD1000/5004 assignment report

Student name: Austin Delic
Curtin student ID: 22930121
Assignment: Introduction to Software Engineering, ISAD1000/5004 2026 Semester 1
Repository name for submission: `Delic_Austin22930121_ISErepo`

1.1. Introduction

I built a Python command-line tool for the assignment scenarios. It finds seasons for selected countries, compares seasons between two countries, compares temperature readings with city averages, and compares readings with Perth's averages.

All file paths in this report are relative to the submitted folder. The code is in `code/`, and the documents are in `documents/`. The program uses Python standard-library modules only, so it can run with raw `python3` commands in a Linux command-line environment.

1.2. Requirements covered

The tool covers these functional requirements from the assignment scenarios:

Requirement	Implemented behaviour
Find a meteorological season	`get_meteorological_season(country, month)` returns the season for a supported country and month.
Find a traditional season	`get_traditional_season(country, month)` returns the Noongar season for Australia or `None` when no traditional season is recorded.
Compare two countries	`compare_country_seasons(country_one, country_two, month)` compares meteorological seasons.
Compare with city average	`compare_with_city_average(city, temperature, period)` compares a reading with the selected city's morning or afternoon average.
Compare with Perth average	`compare_with_perth_average(city, temperature, period)` compares a reading with Perth's average for the same period.
Process file input	`process_request_file(input_path, output_path)` reads CSV-style requests and writes line-by-line results.

The design follows these non-functional constraints:

Constraint	Design response
Runs in a Linux command-line environment	Uses Python standard-library modules only.
Handles invalid input	Validation functions raise specific exceptions, and batch mode writes error messages.
Keeps code maintainable	Static data, validation, season logic, temperature logic, file handling, and CLI code are separated.
Keeps tests practical	Core functions use parameters and return values, while file handling and CLI code stay at the edge.
Uses Figure 2 precision	Temperature inputs are accepted to at most one decimal place.

1.3. Data used

The season data comes from Figure 1 supplied in the assignment. The supported countries are Australia, Spain, Japan, Mauritius, Malaysia, and Sri Lanka.

Australia has two calendars in the program. The meteorological calendar is Summer for December to February, Autumn for March to May, Winter for June to August, and Spring for September to November. The Noongar calendar is Birak for December and January, Bunuru for February and March, Djeran for April and May, Makuru for June and July, Djilba for August and September, and Kambarang for October and November.

Spain and Japan use the northern meteorological calendar from Figure 1. Mauritius uses the four seasons shown in Figure 1. Malaysia and Sri Lanka use the monsoon calendar.

The selected temperature data from Figure 2 covers Perth, Adelaide, and Brisbane:

City	Valid range	Morning average	Afternoon average
Perth	0.7C to 46.0C	18.2C	23.0C
Adelaide	−1.0C to 49.0C	16.5C	21.0C
Brisbane	2.6C to 41.7C	21.8C	24.8C

The program accepts `morning`, `evening`, `afternoon`, and `3pm`. It maps `evening`, `afternoon`, and `3pm` to the same afternoon average, because the supplied table uses the 3PM average.

1.4. Production code design - module descriptions

1.4.1. Original module descriptions

Module	Purpose	Imports	Exports	Behaviour	Dependencies	Exceptions/errors
`data.py`	Store assignment data.	No user input. Data is defined in the module.	Dictionaries and dataclasses used by other modules.	Builds month, country, city, and alias tables.	Python `dataclasses`, `typing`.	No direct validation.
`validation.py`	Normalise and validate user-supplied values.	Parameters: country, month, city, period, temperature.	Canonical values or exceptions.	Converts aliases, checks month range, checks temperature format and city range.	`data.py`, `re`.	Raises `InvalidCountryError`, `InvalidMonthError`,

Module	Purpose	Imports	Exports	Behaviour	Dependencies	Exceptions/errors
						`InvalidCityError`, `InvalidPeriodError`, or `InvalidTemperatu`
`seasons.py`	Handle season lookup and country comparison.	Function parameters.	Strings, `None`, or `SeasonComparison`.	Finds seasons and builds comparison messages.	`data.py`, `validation.py`.	Propagates validation errors. Raises `ValueError` when traditional season output is requested for a non-Australian country.
`temperature.py`	Handle temperature comparisons.	Function parameters.	`TemperatureComparison`.	Classifies readings as same, above, or below average and adds the extra message when the difference is greater than 6.0C.	`data.py`, `validation.py`.	Propagates validation errors.
`io_handlers.py`	Handle batch file requests.	Input and output file paths. File lines use CSV-style commands.	Output text file and processed-line count.	Dispatches request commands to season and temperature functions.	`csv`, `pathlib`, production modules.	Converts validation errors into `ERROR:` output lines.
`main.py`	Provide the command-line interface.	Command-line arguments and keyboard input.	Console output.	Runs demo mode, batch-file mode, or a repeating keyboard menu mode.	`argparse`, production modules.	Prints validation errors in keyboard mode and asks whether I want to ask something else.

1.4.2. Design decisions

I designed the modules to have high cohesion and low coupling. Each module has one main responsibility. Static data is separate from the algorithms, so Figure 1 and Figure 2 values can change without editing comparison logic.

Validation is centralised to avoid repeating the same checks across season, temperature, and file-handling code. The season and temperature modules return structured values instead of printing directly, which makes them easier to test. Console and file output stay in `main.py` and `io_handlers.py`.

The test design covers the main input and output styles: parameter input and return values, keyboard input through the CLI, file input/output through batch mode, exceptions for invalid values, and console output through demo and keyboard mode.

1.5. Production code implementation

1.5.1. Run commands

Run the demo:

```
python3 code/main.py --demo
```

Run batch-file mode:

```
python3 code/main.py --file code/sample_requests.txt code/sample_results.txt
```

Run the keyboard menu:

```
python3 code/main.py
```

After each keyboard-menu request, the program asks `Do you want to ask something else? (y/n):`. Entering `y` or `yes` continues the menu; any other answer stops it.

The program runs with `python3` directly.

1.5.2. Sample production output

Demo output:

```
Season and temperature learning tool demo
-----------------------------------------
Australia in January: Summer
The traditional season in Australia in August is Djilba.
Australia and Japan have different meteorological seasons in January: Australia has Summer, Japan has
Winter.
29.0C is above Perth's afternoon average of 23.0C by 6.0C.
31.0C is above Perth's morning average of 18.2C by 12.8C. The difference is more than 6.0C.
```

Batch-file output:

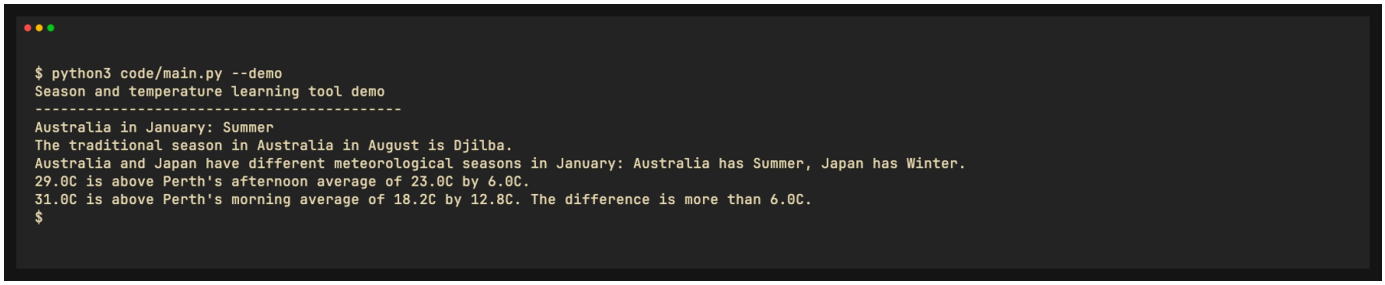
```
Processed 8 request lines.
```

The file `code/sample_results.txt` shows the request results and invalid-data messages.

The batch-file commands are:

Command	Format
Meteorological season	`meteorological,country,month`
Australia Noongar season	`traditional,Australia,month`
Compare countries	`compare,country_one,country_two,month`
City average temperature	`temp,city,period,temperature`
Perth average comparison	`perth,city,period,temperature`

Production demo screenshot:



Batch file run screenshot:

1.5.3. Screenshot and command evidence

The commands above are recorded as text output in this report. Screenshots are stored in `documents/screenshots/` for the final submission package:

Evidence	Command or screen	Screenshot file
Production demo	`python3 code/main.py --demo`	`documents/screenshots/production_demo.png`
Batch file run	`python3 code/main.py --file code/sample_requests.txt code/sample_results.txt`	`documents/screenshots/batch_file_run.png`
Test execution	`python3 -m unittest discover -s code/tests -t code -v`	`documents/screenshots/test_execution.png`
Git log	`git log --oneline --decorate --graph --all`	`documents/screenshots/git_log.png`

1.5.4. Modularity checklist

No.	Checklist question	Design concept
1	Does the module have one main task?	Cohesion
2	Does the module avoid doing unrelated sequential tasks?	Cohesion
3	Does the module avoid control flags that select unrelated behaviour?	Coupling and cohesion
4	Does the module avoid global mutable variables for data flow?	Coupling
5	Are imports and exports clear from parameters, return values, files, or console use?	Module descriptions
6	Are parameter lists short enough to understand?	Coupling
7	Is repeated logic placed in one module?	Redundancy and reuse
8	Can the module be tested without relying on manual input?	Testability
9	Are exceptions or error messages used consistently for invalid data?	Error handling
10	Can a likely future data change be made without editing unrelated modules?	Maintenance

1.5.5. Review results

Module	Cohesion review	Coupling review	Redundancy review	Decision
`data.py`	One task: store figure data.	Other modules import constants only.	Shared calendars reuse `_calendar`.	Keep.
`validation.py`	One task: validation and normalisation.	Depends on data tables only.	Avoids repeating validation logic elsewhere.	Keep.
`seasons.py`	One task: season lookup and comparison.	Uses validation and data through clear calls.	Builds season messages in one place and avoids file-handler formatting duplication.	Keep.
`temperature.py`	One task: temperature comparison.	Uses validation and city data through clear calls.	`_validated_reading` and `_build_comparison` remove repeated validation and same/above/below logic.	Keep.
`io_handlers.py`	One task: batch request processing.	Calls public production functions.	Command dispatch is local and delegates message formatting to production functions.	Keep.
`main.py`	One task: CLI flow.	Calls public production functions.	Demo and keyboard paths are separate by purpose.	Keep.

1.5.6. Refactoring result

The refactor replaced earlier sample data with the supplied Figure 1 and Figure 2 values. Temperature validation now accepts at most one decimal place and rejects readings outside the selected city's range. Repeated alias lookup, meteorological message formatting, and temperature input preparation moved into small helper functions. The CLI and production logic were already loosely coupled, so a full rewrite was not needed.

Original issue	Refactor applied	Design concept	Result
Season and temperature values were initially close to sample/demo data.	Figure 1 and Figure 2 tables replaced them in `data.py`.	Maintenance and low coupling	Data changes stay in one module instead of spreading through the algorithms.
Temperature validation could have been repeated in city comparison and Perth comparison code.	Numeric parsing, precision checking, and city range checking moved into `validation.py`.	Redundancy and reuse	Both comparison functions use the same validation rules.
Country and city alias lookup had the same structure.	One private lookup helper was added in `validation.py`, while `normalize_country()` and `normalize_city()` remain the public validation functions.	Redundancy and cohesion	Duplicate alias-checking code was removed without changing the public module interface.
Meteorological batch output was formatted inside file handling.	`format_meteorological_season()` now lives in `seasons.py`, and `io_handlers.py` calls it.	Cohesion and low coupling	Season message formatting now belongs with the season logic instead of the file input module.
Same/above/below message construction could have been duplicated.	`_build_comparison()` was added in `temperature.py`.	Cohesion and redundancy	The public functions choose the relevant average, then reuse one comparison builder.
City, period, temperature, and range validation repeated across temperature comparison functions.	`_validated_reading()` was added in `temperature.py`.	Redundancy and reuse	Both temperature comparison functions now share one input-preparation path.
CLI, file handling, and core logic could have been mixed together.	`main.py` and `io_handlers.py` stay at the input/output edge, and season/temperature logic stays parameter-based.	Coupling and testability	Core behaviour can be tested without manual keyboard input.

1.5.7. Revised module descriptions

The revised module descriptions remain in the table under “Original module descriptions”. Module responsibilities stayed the same after refactoring. Data and validation details changed to match the supplied figures.

1.6. Black-box test cases

1.6.1. Equivalence partitioning

Module/function	Category	Test data	Expected result	Test code
`get_meteorological_season`	Supported southern country	Australia, January	Summer	`test_australia_meteorological_seasons`
`get_meteorological_season`	Supported northern country	Spain, March	Spring	`test_northern_countries_share_meteorological_ca`
`get_meteorological_season`	Supported custom country	Mauritius, October	Spring	`test_mauritius_custom_meteorological_calendar`
`get_meteorological_season`	Supported monsoon country	Malaysia, July	Southeast Monsoon	`test_malaysia_and_sri_lanka_monsoon_calendar`
`get_meteorological_season`	Unsupported country	Canada, March	`InvalidCountryError`	`test_unsupported_country_is_rejected`
`get_traditional_season`	Traditional data exists	Australia, August	Djilba	`test_australia_traditional_noongar_season`
`get_traditional_season`	Traditional data does not exist	Spain, January	`None`	`test_country_without_traditional_calendar_retur`
`format_traditional_season`	Traditional output is Australia-only	Spain, January	`ValueError`	`test_country_without_traditional_calendar_retur`
`compare_country_seasons`	Same season	Malaysia, Sri Lanka, July	`same` is true	`test_compare_same_meteorological_season`
`compare_country_seasons`	Different season	Australia, Japan, January	`same` is false	`test_compare_different_meteorological_season`
`compare_with_city_average`	Same as average	Perth, 23.0, afternoon	Relation is same	`test_same_as_average`
`compare_with_city_average`	Above average	Brisbane, 31.0, morning	Relation is above	`test_above_average_with_large_difference`
`compare_with_city_average`	Selected Figure 2 cities	Perth, Adelaide, Brisbane	Only three city profiles are loaded	`test_selected_city_profiles_are_limited_to_thre`
`compare_with_city_average`	Below average	Adelaide, 20.0, afternoon	Relation is below	`test_below_average`
`compare_with_city_average`	Invalid city	Melbourne, 20.0, morning	`InvalidCityError`	`test_invalid_city`
`compare_with_city_average`	Invalid period	Perth, 20.0, night	`InvalidPeriodError`	`test_invalid_period`
`compare_with_city_average`	Invalid numeric range	Adelaide, 121, morning	`InvalidTemperatureError`	`test_last_three_student_id_digits_temperature_i`
`process_request_line`	Known command	`meteorological,Australia,January`	Season message	`test_meteorological_request_line`
`process_request_line`	Unknown command	`unknown,Australia,January`	`ERROR:` message	`test_unknown_command_reports_error`

The required numeric data `121` is tested as an invalid Adelaide temperature.

1.6.2. Boundary value analysis

Module/function	Boundary	Test data	Expected result	Test code
`parse_month` through season lookup	Minimum valid month	`1`	Accepted	`test_invalid_month_boundaries` plus valid month tests
`parse_month` through season lookup	Maximum valid month	`12`	Accepted	valid month tests
`parse_month` through season lookup	Just below minimum	`0`	`InvalidMonthError`	`test_invalid_month_boundaries`
`parse_month` through season lookup	Just above maximum	`13`	`InvalidMonthError`	`test_invalid_month_boundaries`
`compare_with_city_average`	Perth minimum valid temperature	`0.7`	Accepted	`test_minimum_and_maximum_city_boundaries_are_va`
`compare_with_city_average`	Perth maximum valid temperature	`46.0`	Accepted	`test_minimum_and_maximum_city_boundaries_are_va`
`compare_with_city_average`	Just below Perth minimum	`0.6`	`InvalidTemperatureError`	`test_outside_city_range_is_invalid`
`compare_with_city_average`	Just above Perth maximum	`46.1`	`InvalidTemperatureError`	`test_outside_city_range_is_invalid`
`_build_comparison` through public function	Extra-message threshold	Difference `6.0C`	No extra message	`test_difference_of_exactly_six_is_not_large`
`_build_comparison` through public function	Above threshold	Difference more than `6.0C`	Extra message	`test_above_average_with_large_difference`

1.7. White-box test cases

Module/function	Construct	Path	Test data	Expected result	Test code
`compare_country_seasons`	`if/elif/else`	Meteorological branch	Australia, Japan, January	Different seasons	`test_compare_different_meteorological_sea`

Module/function	Construct	Path	Test data	Expected result	Test code
`format_traditional_season`	Exception path	Non-Australia request	Spain, January	`ValueError`	`test_country_without_traditional_calendar`
`_build_comparison`	`if`	Same branch	Perth, 23.0, afternoon	Relation is same	`test_same_as_average`
`_build_comparison`	`elif`	Above branch	Brisbane, 31.0, morning	Relation is above	`test_above_average_with_large_difference`
`_build_comparison`	`else`	Below branch	Adelaide, 20.0, afternoon	Relation is below	`test_below_average`
`_build_comparison`	Boolean condition	Difference equals 6.0C	Perth, 29.0, afternoon	No extra message	`test_difference_of_exactly_six_is_not_lar`
`_build_comparison`	Boolean condition	Difference is greater than 6.0C	Brisbane, 31.0, morning	Extra message	`test_above_average_with_large_difference`
`process_request_line`	Command dispatch	Known commands	meteorological, traditional, compare, temp, perth	Correct messages	file-handler tests
`process_request_line`	Error path	Unknown command	unknown	`ERROR:` message	`test_unknown_command_reports_error`
`process_request_line`	Exception path	Validation error	Canada country	`ERROR:` message	`test_validation_error_is_written_as_error`
`main`	Command-line branch	`--demo` branch	`--demo`	Demo heading and sample output	`test_main_demo_branch_prints_demo_output`
`main`	Command-line branch	`--file` branch	Request and result file paths	Processed-line count and output file	`test_main_file_branch_processes_file`
`run_interactive`	`if/elif/else`	Unknown option branch	input `9`	`Unknown option.`	`test_interactive_unknown_option_path`
`run_interactive`	Exception path	Validation error branch	option `1`, country `Canada`, month `March`	Error message	`test_interactive_validation_error_path`
`run_interactive`	Loop and boolean condition	Ask another question branch	answer `y`, then answer `n`	Second request runs, then menu exits	`test_interactive_can_run_another_request`

These tests cover the main path types in my production code: `if`, `elif`, `else`, boolean conditions, loops, command-dispatch branches, command-line branches, and expected exceptions. `test_process_request_file_writes_results` covers the file-processing loop.

1.8. Test implementation and execution

The tests use Python's `unittest` framework and are stored in `code/tests/`.

Run tests:

```
python3 -m unittest discover -s code/tests -t code -v
```

The tests run with `python3` directly.

Execution result:

```
Ran 41 tests in 0.002s

OK
```

Test execution screenshot:

```
$ python3 -m unittest discover -s code/tests -t code -v
test_blank_and_comment_lines_are_skipped (tests.test_io_handlers.FileHandlerTests) ... ok
test_compare_request_line (tests.test_io_handlers.FileHandlerTests) ... ok
test_compare_request_rejects_old_extra_field_format (tests.test_io_handlers.FileHandlerTests) ... ok
test_meteorological_request_line (tests.test_io_handlers.FileHandlerTests) ... ok
test_perth_request_line (tests.test_io_handlers.FileHandlerTests) ... ok
test_process_request_file_writes_results (tests.test_io_handlers.FileHandlerTests) ... ok
test_temperature_request_line (tests.test_io_handlers.FileHandlerTests) ... ok
test_traditional_request_line (tests.test_io_handlers.FileHandlerTests) ... ok
test_traditional_request_rejects_non_australia (tests.test_io_handlers.FileHandlerTests) ... ok
test_unknown_command_reports_error (tests.test_io_handlers.FileHandlerTests) ... ok
test_validation_error_is_written_as_error (tests.test_io_handlers.FileHandlerTests) ... ok
test_interactive_can_run_another_request (tests.test_main.MainCliTests) ... ok
test_interactive_compare_is_meteorological_only (tests.test_main.MainCliTests) ... ok
test_interactive_unknown_option_path (tests.test_main.MainCliTests) ... ok
test_interactive_validation_error_path (tests.test_main.MainCliTests) ... ok
test_main_demo_branch_prints_demo_output (tests.test_main.MainCliTests) ... ok
test_main_file_branch_processes_file (tests.test_main.MainCliTests) ... ok
test_australia_meteorological_seasons (tests.test_seasons.SeasonLookupTests) ... ok
test_australia_traditional_noongar_season (tests.test_seasons.SeasonLookupTests) ... ok
test_compare_different_meteorological_season (tests.test_seasons.SeasonLookupTests) ... ok
test_compare_same_meteorological_season (tests.test_seasons.SeasonLookupTests) ... ok
test_country_without_traditional_calendar_returns_none (tests.test_seasons.SeasonLookupTests) ... ok
test_invalid_month_boundaries (tests.test_seasons.SeasonLookupTests) ... ok
test_malaysia_and_sri_lanka_monsoon_calendar (tests.test_seasons.SeasonLookupTests) ... ok
test_mauritius_custom_meteorological_calendar (tests.test_seasons.SeasonLookupTests) ... ok
test_northern_countries_share_meteorological_calendar (tests.test_seasons.SeasonLookupTests) ... ok
test_unsupported_country_is_rejected (tests.test_seasons.SeasonLookupTests) ... ok
test_above_average_with_large_difference (tests.test_temperature.TemperatureComparisonTests) ... ok
test_below_average (tests.test_temperature.TemperatureComparisonTests) ... ok
test_boolean_temperature_is_invalid (tests.test_temperature.TemperatureComparisonTests) ... ok
test_compare_reading_with_perth_average (tests.test_temperature.TemperatureComparisonTests) ... ok
test_difference_of_exactly_six_is_not_large (tests.test_temperature.TemperatureComparisonTests) ... ok
test_evening_and_3pm_map_to_afternoon_average (tests.test_temperature.TemperatureComparisonTests) ... ok
test_invalid_city (tests.test_temperature.TemperatureComparisonTests) ... ok
test_invalid_period (tests.test_temperature.TemperatureComparisonTests) ... ok
test_last_three_student_id_digits_temperature_is_invalid_for_adelaide (tests.test_temperature.TemperatureComparisonTests) ... ok
test_minimum_and_maximum_city_boundaries_are_valid (tests.test_temperature.TemperatureComparisonTests) ... ok
test_outside_city_range_is_invalid (tests.test_temperature.TemperatureComparisonTests) ... ok
test_same_as_average (tests.test_temperature.TemperatureComparisonTests) ... ok
test_selected_city_profiles_are_limited_to_three_cities (tests.test_temperature.TemperatureComparisonTests) ... ok
test_temperature_with_two_decimal_places_is_invalid (tests.test_temperature.TemperatureComparisonTests) ... ok

----------------------------------------------------------------------
Ran 41 tests in 0.003s

OK
$
```

The tests cover these testing concerns:

Testing issue	Implementation
Production code and test code are separate	Production code is in `code/season_tool/`, and tests are in `code/tests/`.
Tests are repeatable	Tests run without manual input.
Parameters and return values are tested	Season and temperature functions are called directly.
Exceptions are tested	`assertRaises` checks invalid country, month, city, period, and temperature values.
File input/output is tested	`tempfile.TemporaryDirectory()` creates isolated request and result files.
Console output is tested	`redirect_stdout` captures demo, file-mode, repeat-prompt, and interactive output.
Test fixtures are used where needed	`tempfile` creates and removes temporary files.

The first test run showed that `unittest` discovery needed `code/tests/__init__.py` when using `-t code`. After adding that file, the suite passed.

1.9. Summary of my work (traceability matrix)

Module name	BB EP	BB BVA	WB	Data types	Form of input/ output	EP code	BVA code	WB code
`data.py`	done	not separate	not separate	strings, integers, floats	imported constants	covered through other modules	covered through other modules	covered through other modules
`validation.py`	done	done	partial	strings, integers, floats, boolean	parameters, exceptions	done	done	done
`seasons.py`	done	done	done	strings, integers, boolean result	parameters, return values	done	done	done
`temperature.py`	done	done	done	strings, floats, boolean result	parameters, return values	done	done	done
`io_handlers.py`	done	partial	done	strings, file paths	text file input, text file output	done	partial	done
`main.py`	partial	not separate	done	strings	keyboard input, command-line arguments, console output	CLI output tests	core BVA covered through modules	`test_main.py` covers demo, file, interactive option, repeat-prompt, and validation-error paths

1.10. Version control

Git was used locally.

The submission repository's name:

`Delic_Austin22930121_ISErepo`

The branch plan was:

Branch	Purpose	Merge point
`main`	Completed work and submission-ready files live here.	Completed feature branches were merged into it.
`production-code`	Season, temperature, validation, CLI, and file-processing modules were developed here.	Merged after syntax checks and demo execution.
`testing`	Black-box and white-box test code was developed here.	Merged after all tests passed.
`report-docs`	Report, README, traceability matrix, and evidence files were developed here.	Merged after recording command output and the Git log.

Commits were made after these milestones:

- planning and repository structure
- production module implementation
- sample data and demo entry point
- test implementation
- report and traceability matrix
- verification output updates

Separate commits show the work moving through planning, coding, testing, and documentation instead of one final commit.

Season data came from Figure 1 supplied for the assignment. The selected Figure 2 city data covers Perth, Adelaide, and Brisbane. Perth was kept because the scenario requires comparison with Perth's average.

Version-control concepts used in this project:

Concept	Use in this project
Working directory	Files were edited in the project folder.
Staging area	Related files were staged before each commit.
Commits	Work was committed after planning, production code, tests, documentation, and PDF generation.
Branching	Separate branches kept production, testing, and documentation work apart.
Merging	Feature branches were merged back into `main`.
Log	`git log --oneline --decorate --graph --all` shows the history.

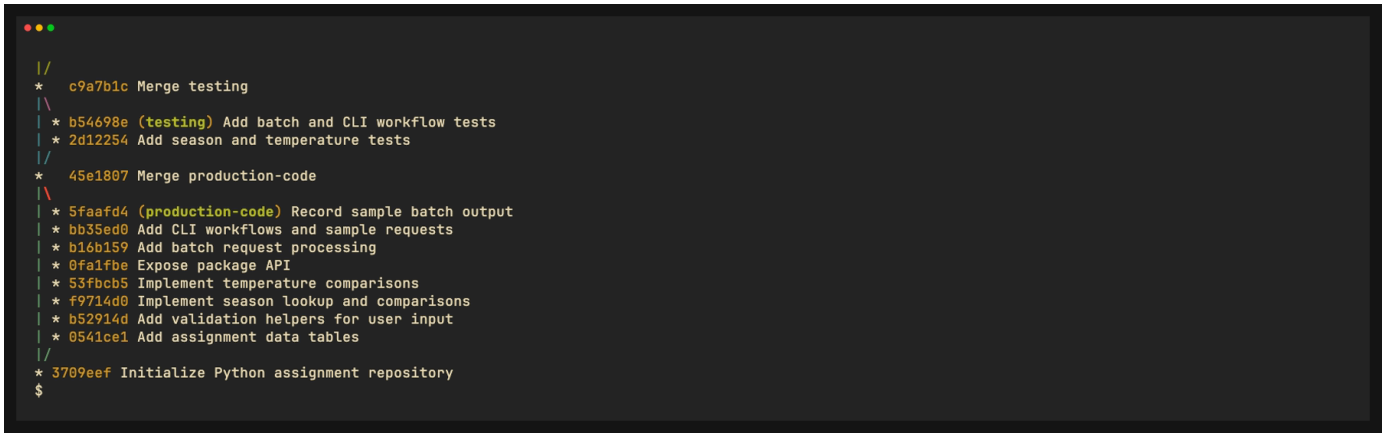
Useful log command:

```
git log --oneline --decorate --graph --all
```

Other evidence commands:

```
git status
git diff
```

Git log screenshot:



1.11. Discussion

My project meets the two assignment scenarios for the selected Figure 1 countries and the Figure 2 cities. It has parameter-based functions, keyboard input, file input, return values, console output, and file output. The test suite covers valid values, invalid values, boundaries, branches, exceptions, and file handling.

The hardest part was keeping the design small while still showing the required testing skills. I put the core logic in testable functions and left file and keyboard handling at the edge of the program.

The main limitation is my fixed data set. A better version would load country and city data from a CSV or JSON file. I would also add a menu option that lists supported countries and cities before the user enters data.

The final test suite includes small CLI tests that capture console output for demo, file, repeat-prompt, and interactive menu paths.

1.12. Submission checklist

Item	Status
Code files are inside `code/`	done
Documents are inside `documents/`	done
Markdown report is present	done
PDF report is present	done
README explains submitted files	done
`.git` directory is present in repository	done
Tests pass with `python3`	done
Screenshots or copied command evidence added to final PDF	done